

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

ESPECIALIDAD DE INGENIERÍA INFORMÁTICA



Proyecto de Diseño y Desarrollo de Software
[1INF54-0982]

Estándar de Control de Versiones

Equipo: 6F

Integrantes:

- Gabriel Angel Quispe Cerron
- Roly Alvaro Huacre Samaniego
- Amira Paola Rodriguez Almora
- Diego Alfredo Carmona Chumbimuni

2026-2

Lima, 15 de septiembre

Historial de versiones

Fecha	Versión	Descripción	Autor
15/09/2026	1.0	Se ha redactado la primera versión de los estándares de programación web frontend.	Equipo 6F

Contenido

1. Introducción.....	4
2. Estructura del nombre de una rama.....	4
3. Convenciones de commits.....	4
4. Política de Squash and Merge.....	5
5. Resolución técnica de conflictos en el monorepo.....	5
5.1. Prevención.....	5
5.2. Resolución.....	6
5.3. Verificación post-resolución.....	6
6. Gestión y Revisión de Pull Requests (PR).....	6
7. Protección de Ramas y Automatización.....	6
8. Versionado y Entregables (Releases).....	7
9. Referencias y recursos.....	7

1. Introducción

El Plan de Gestión de la Configuración (PGC) del proyecto establece las reglas generales de gobierno sobre el repositorio. Este documento complementa al PGC con la guía técnica que todo desarrollador debe seguir día a día: cómo nombrar una rama, cómo integrar sus cambios y cómo resolver conflictos dentro del monorepo de SysMile, el cual contiene el backend Java, el frontend web y los scripts de infraestructura en un único repositorio.

2. Estructura del nombre de una rama

Toda rama de trabajo debe crearse a partir de la rama *main* y seguir el formato *tipo/{ticket}-descripcion-corta*, en minúsculas y con palabras separadas por guiones:

Prefijo	Uso	Ejemplo
feature/	Nueva funcionalidad	feature/SYS-142-alns-destroy-operator
fix/	Corrección de un defecto	fix/SYS-201-fix-route-eta-calculation
hotfix/	Corrección urgente sobre producción	hotfix/SYS-305-fix-auth-token-expiry
refactor/	Reestructuración sin cambio de comportamiento	refactor/SYS-118-extract-route-mapper
chore/	Tareas de mantenimiento (dependencias, configuración)	chore/SYS-090-upgrade-spring-boot
docs/	Cambios exclusivos de documentación	docs/SYS-050-update-api-standard

- El identificador de ticket (*{ticket}*) debe corresponder siempre a una tarea existente en el tablero del proyecto; no se permiten ramas sin ticket asociado.
- Evitar crear ramas con descripciones mayores a 30 caracteres; se debe priorizar la claridad sobre la exhaustividad.
- Las ramas deben tener una vida corta (idealmente menos de 3 días) para reducir la probabilidad de conflictos con *main*.

3. Convenciones de commits

Se utiliza *Conventional Commits* con el ticket incluido, bajo el formato: *tipo({ticket}): descripción en minúsculas y en modo imperativo*.

None

```
feat(SYS-142): agregar operador de destrucción aleatoria a ALNS
fix(SYS-201): corregir cálculo de ETA cuando la ruta no tiene
paradas
refactor(SYS-118): extraer el mapeo de Route a RouteResponseDTO
```

- Tipos permitidos: *feat*, *fix*, *refactor*, *docs*, *test*, *chore*, *perf* y *style*.
- Un commit debe representar un cambio lógico único; se deben evitar commits que mezclen refactorización con nueva funcionalidad.

4. Política de Squash and Merge

- Toda integración de una rama de tipo *feature*/ o *fix*/ hacia *main* se realiza mediante *Squash and Merge*, evitado el uso de *merge commit* tradicional o *fast-forward* directo.
- El mensaje del commit resultante del *squash* debe reescribirse siguiendo el formato de *Conventional Commits* descrito en la sección 3, resumiendo el conjunto de cambios de la rama y sin concatenar automáticamente todos los mensajes individuales.
- Excepción: las ramas *hotfix*/ que deban aplicarse simultáneamente sobre *main* y una rama de *release* emplean *cherry-pick* del commit previamente procesado con *squash*, evitando un nuevo *squash* independiente para mantener una trazabilidad idéntica en ambas ramas.
- Antes de aprobar el *squash*, el autor de la rama debe realizar un *rebase* interactivo local para eliminar commits temporales (como "wip" o "fix typo"), garantizando un historial local limpio que facilite la revisión.
- Está prohibido el uso de *force-push* sobre *main*. La opción *force-push* (*--force-with-lease*) solo se permite sobre ramas de *feature* propias y nunca sobre ramas compartidas por múltiples desarrolladores.

5. Resolución técnica de conflictos en el monorepo

Al tratarse de un monorepo que integra backend, frontend e infraestructura, los conflictos suelen originarse en archivos de configuración compartidos (*pom.xml*, *package.json*, *docker-compose.yml*) y en contratos de API consumidos por ambos lados. Para ello, se establecen las siguientes pautas:

5.1. Prevención

- Sincronizar la rama de trabajo con *main* mediante *rebase* (no *merge*) al menos una vez al día, con el fin de detectar conflictos de manera temprana en lotes pequeños.
- Evitar que una misma rama modifique simultáneamente archivos de configuración de construcción (*pom.xml*, *package.json*) y código funcional; se deben separar estos cambios en commits o ramas distintas cuando sea posible.

5.2. Resolución

- Al resolver conflictos mediante *git rebase main*, se debe proceder commit por commit y ejecutar la suite de pruebas relevante (backend o frontend, según el módulo afectado) antes de continuar con *git rebase --continue*.
- Para conflictos en archivos de dependencias (*package-lock.json*, *pom.xml*), no se deben fusionar manualmente los bloques de versiones. Se debe tomar la versión de *main* como base y volver a instalar o agregar la dependencia sobre dicha base para evitar inconsistencias en el árbol de dependencias.
- En caso de conflictos en contratos de API compartidos (esquemas DTO usados como referencia para el frontend), se debe coordinar explícitamente con el responsable del otro módulo antes de resolver la discrepancia, priorizando siempre la versión que cumpla con el estándar de diseño de APIs vigente (documento *65.std.api*).
- Se prohíbe resolver un conflicto eliminando código de otra rama sin comprender su propósito. En caso de duda, se debe marcar el bloque con un comentario *TODO(CONFLICTO)* y solicitar una revisión antes de continuar.

5.3. Verificación post-resolución

- Tras resolver cualquier conflicto, se debe ejecutar localmente la compilación completa del monorepo (backend y frontend) antes de publicar los cambios, dado que un conflicto mal resuelto en un módulo puede romper la compilación del otro si comparten contratos.

6. Gestión y Revisión de Pull Requests (PR)

- **Plantilla Obligatoria:** Todo Pull Request debe crearse utilizando una plantilla estandarizada en el repositorio. La descripción debe incluir: el identificador y enlace al ticket, un resumen claro de los cambios técnicos implementados, instrucciones paso a paso para probar la funcionalidad localmente y, en caso de modificaciones en la interfaz (frontend), capturas de pantalla o un breve video demostrativo.
- **Políticas de Aprobación:** Queda estrictamente prohibido que un autor apruebe su propio código. Para que un PR sea elegible para su integración (Merge) hacia la rama *main*, requiere obligatoriamente la revisión técnica y la aprobación formal (Approve) de al menos un desarrollador del equipo distinto al autor original.

7. Protección de Ramas y Automatización

- **Reglas de Protección (Branch Protection):** La rama *main* debe configurarse como protegida en la plataforma de alojamiento del repositorio. Queda bloqueado cualquier push directo a *main*; toda adición de código debe transitar obligatoriamente por el flujo de Pull Requests.
- **Verificaciones de Estado (Status Checks):** El botón de integración (Merge) permanecerá bloqueado hasta que el código propuesto pase exitosamente todas las integraciones continuas requeridas: compilación completa del monorepo, paso exitoso de la suite de pruebas y validación estática de código.
- **Git Hooks (Pre-commit):** Se exige el uso de Git Hooks locales (implementados a través de herramientas como Husky y lint-staged) para ejecutar tareas de limpieza de forma automática. El flujo debe abortar el intento de *git commit* si los archivos modificados no cumplen con las reglas de formateo de Prettier o presentan errores en los linters del frontend o backend.

8. Versionado y Entregables (Releases)

- **Versionado Semántico:** El proyecto SysMile se adhiere al estándar de Versionado Semántico (SemVer) bajo el formato vMAJOR.MINOR.PATCH (ej. v1.2.0) para identificar los estados estables y liberaciones del sistema.
- **Etiquetado de Versiones (Tags):** Cuando el código acumulado en la rama *main* se considere estable y listo para el entorno de producción, se debe crear un tag oficial en Git. Este tag actuará como la fuente de verdad histórica de la versión y será el disparador exclusivo para la construcción final de las imágenes de Docker y el despliegue a producción.

9. Referencias y recursos

Conventional Commits: <https://www.conventionalcommits.org/>

Documentación oficial de Git: <https://git-scm.com/doc>

GitHub Flow: <https://docs.github.com/en/get-started/using-github/github-flow>